EXPERIMENT 14

Pro-Code Extension of a Zoho Creator Application

Exploring Deluge custom functions and Zoho Catalyst through an illustrative priority-scoring workflow

241401068

Submitted by	PRANAV SRINIVAS
Register number	241401068
Application	Patient Appointment Scheduling System

PRACTICAL STUDY AND IMPLEMENTATION PLAN

1. Aim and scope

This experiment explores extending a Zoho Creator application through reusable Deluge custom functions and a serverless function in Zoho Catalyst. Appointment priority scoring serves as an example of how Creator can send data to an external service and use the response. The sample code is illustrative, not a record of a deployed implementation.

2. Procedure

Step	Activity
1. Select a requirement	Use appointment priority scoring as an example of logic beyond a simple workflow action.
2. Compare options	Review workflow actions, Deluge custom functions, Creator Custom APIs, and Catalyst functions.
3. Design the architecture	Map the Creator form and workflow to a Catalyst function through a managed connection.
4. Prepare the Creator function	Package input as JSON and call the endpoint using invokeurl.
5. Prepare Catalyst logic	Receive the payload, apply example rules, and return a JSON priority value.
6. Plan failure handling	Define the success path, error path, and criteria for choosing an extension type.

3. Output — architecture

Zoho Creator — low-code	Zoho Catalyst — pro-code
Patient form → workflow → Deluge custom function → managed connection and HTTPS request	Advanced I/O function → validate input → apply business logic → return JSON response
Creator reads the returned priority and updates the relevant record.	Optional storage or external API can support additional processing.

Creator retains the form and workflow. Catalyst handles logic that may benefit from a general-purpose language or external libraries. The response is returned to Creator for record updating.

3.1 Extension options compared

Option	Description	Skill	Typical use
Workflow actions	Built-in actions such as notifications and record updates.	Low	Routine automation.
Deluge custom function	Reusable function called from Creator workflows.	Low–medium	Shared business rules.
Creator Custom API	REST endpoint exposed by a Creator application.	Medium	External systems calling Creator.
Catalyst function	Serverless Node.js, Java, or Python function invoked over HTTPS.	Medium–high	Complex logic, libraries, external systems.

24140168

3.2 Deluge custom function — illustrative

A reusable Deluge function can package the age and condition fields into a payload, call the Catalyst endpoint using a managed connection, and return the priority value from the response. A workflow can then store the result in the relevant record.

The endpoint and connection names in the supplied example are placeholders. The request format, authentication, response shape, and failure behaviour need to be confirmed and tested before deployment.

3.3 Catalyst function — illustrative scoring

The sample Node.js logic reads age and existing conditions from a JSON request. It assigns example points for age 60 or above, a heart-disease entry, and a diabetes entry, then maps the total to High, Medium, or Normal. These thresholds are only programming examples and are not clinical guidance. Real medical prioritisation rules require approval from qualified medical staff.

3.4 Execution flow

Submit	Call	Score	Validate response	Update
Patient form is submitted.	Creator workflow calls the Deluge function.	Catalyst validates input and calculates the example score.	Check for a valid response or failure.	Update the priority when valid; log errors otherwise.

A failed external request should not cause the appointment itself to be lost. The design therefore describes retaining a default priority and recording an error when a response is unavailable. A real implementation should include timeouts, response validation, and appropriate error handling.

3.5 Choosing an extension

Question	Direction
Can a built-in action solve it?	Prefer a standard workflow action for simplicity.
Is the business rule reusable within Creator?	Use a Deluge custom function.
Must an outside system call Creator?	Consider a Creator Custom API.
Are libraries, heavy computation, or external systems needed?	Consider a Catalyst function and review security and operational needs.
No option fits well?	Reconsider and simplify the requirement if possible.

3.6 Benefits and precautions

Aspect	Benefit	Precaution
Flexibility	General-purpose languages and libraries support specialised logic.	More code requires testing and maintenance.
Reuse	One function can serve multiple workflows.	Keep inputs and outputs consistent.
Separation	External processing can keep Creator workflows lighter.	Handle network errors and timeouts.
Security	Managed connections keep credentials out of scripts.	Restrict access and validate inputs.
Limits and cost	Serverless infrastructure can scale execution.	Check current service limits and pricing.

4. Final result

The study describes a two-part extension pattern: a reusable Deluge function within Creator and a Catalyst serverless function reached through a managed HTTPS connection. It compares the available extension choices, outlines an illustrative scoring exchange, and identifies security and error-handling precautions. The source experiment states that the code was not deployed or tested, so no successful live run is claimed.

24140068